

CS3423 Mini Assignment 2

Q1: “warning, rule cannot be matched”

CS24BTECH11030, CS24BTECH11036

1 When does this warning appear?

Flex compiles all the rules in a .l file into a single DFA and, for a given input, always picks the rule that matches the *longest* prefix of the remaining input. If two or more rules would match the same (longest) prefix, the rule that appears *first* in the file wins.

The warning

```
warning, rule cannot be matched
```

is printed when flex can statically prove that a rule will *never* be selected under this longest-match / first-rule-wins policy — i.e. every string the rule’s pattern can match is already matched by some earlier rule with a length at least as long. In that case the later rule’s action is dead code, since control never reaches it, so flex warns at compile time instead of leaving a silent bug.

This typically happens when:

1. An earlier pattern’s language is a *superset* of a later pattern’s language (e.g. a general identifier rule placed before a keyword rule that is really just a special case of it).
2. Two patterns match strings of exactly the same length on some inputs, and the earlier one already matches every such string, so the later, more specific rule never gets the chance to fire on the tie.

2 Example 1 — keyword shadowed by an earlier general rule

```
%%  
[a-z]+ { printf("WORD\n"); }  
"end" { printf("END\n"); }  
%%  
int main(void) { yylex(); return 0; }  
int yywrap(void) { return 1; }
```

Every string matched by "end" (namely the single string "end") is also matched by [a-z]+, and for that string both rules match the same length (3 characters). Since [a-z]+ appears first, it always wins the tie, so the "end" rule can never fire.

Compiling this file with flex gives:

```
$ flex example1.l  
example1.l:3: warning, rule cannot be matched
```

3 Example 2 — a specific rule subsumed by a general one

```
%%  
[0-9]+ { printf("NUM\n"); }  
[0-9]{3} { printf("THREE_DIGITS\n"); }  
%%  
int main(void) { yylex(); return 0; }  
int yywrap(void) { return 1; }
```

`[0-9]{3}` only matches strings of exactly three digits, and every such string is also matched by `[0-9]+` with the same length. Because `[0-9]+` is listed first, it always wins on the tie, so the `[0-9]{3}` rule can never be reached.

Compiling this file with flex gives:

```
$ flex example2.1  
example2.1:3: warning, rule cannot be matched
```

4 Files submitted

- `example1.1` — keyword shadowed by a general identifier rule.
- `example2.1` — fixed-length rule shadowed by an unbounded rule.

Both were verified with `flex` (version 2.6.4) and reproduce the exact warning shown above.